

DSA Interview Q&A (2–4 Years Experience)

Service-Based Company (Real Mix)

1 Question: Find the First Non-Repeating Character in a String

What they asked me

“Given a string, return the first non-repeating character. What’s the time complexity?”

What I said

I used a **frequency map approach**:

Solution Approach

1. First pass → count frequency using a hash map
2. Second pass → find the first char with freq = 1
3. Return it, else return -1

Code

```
def firstNonRepeating(s):  
    freq = {}  
    for ch in s:  
        freq[ch] = freq.get(ch, 0) + 1  
  
    for ch in s:  
        if freq[ch] == 1:  
            return ch  
    return -1
```

Time Complexity

$O(n)$

Tips

- Never loop inside a loop → $O(n^2)$
- Always mention **hash map = optimal for frequency problems**

2 Question: Reverse a Linked List (Iterative)

What they asked me

“Explain how to reverse a linked list. Give iterative solution.”

What I said

I used **three pointers**: prev, curr, next.

Code

```
def reverseList(head):  
    prev = None  
    curr = head  
  
    while curr:  
        nxt = curr.next  
        curr.next = prev  
        prev = curr  
        curr = nxt  
  
    return prev
```

Tips

- Mention: “Iterative = $O(n)$, Recursive = uses extra stack.”
- Interviewers prefer iterative.

3 Question: Two Sum Problem (Optimal)

What they asked me

“Given an array and target, return indices of two numbers that sum to target.”

What I said

I used a **hash map** to store visited elements.

Code

```
def twoSum(nums, target):  
    seen = {}  
    for i, num in enumerate(nums):
```

```
diff = target - num
if diff in seen:
    return [seen[diff], i]
seen[num] = i
```

Time Complexity

$O(n)$

Tips

- Hash map solution always preferred
 - Don't use nested loops ($O(n^2)$)
-

4 Question: Detect Cycle in Linked List (Floyd's Algorithm)

What they asked me

"How do you detect a cycle in a linked list?"

What I said

I used the **slow and fast pointer method**:

Code

```
def hasCycle(head):
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

Tips

- This is the MOST preferred solution for cycle detection
 - Always mention: "Uses $O(1)$ extra space"
-

5 Question: Find Missing Number in $0\dots n$

What they asked me

“An array contains numbers from 0 to n with one missing. Find the missing number.”

What I said

I used the **sum formula approach**:

Code

```
def missingNumber(nums):  
    n = len(nums)  
    total = n * (n + 1) // 2  
    return total - sum(nums)
```

Tips

- Also mention XOR solution → interviewers like it
-

6 Question: Valid Parentheses

What they asked me

“Check if a string has balanced parentheses.”

What I said

Use a **stack**.

Code

```
def isValid(s):  
    stack = []  
    map = {')': '(', ']': '[', '}': '{'}  
  
    for ch in s:  
        if ch in map.values():  
            stack.append(ch)  
        else:  
            if not stack or stack[-1] != map[ch]:  
                return False  
            stack.pop()  
    return len(stack) == 0
```

Tips

- Don't forget `len(stack) == 0` check

- Stack is always the best solution here
-

7 Question: Merge Two Sorted Arrays

What they asked me

“Merge two sorted arrays without using extra space.”

What I said

I used the **Gap method** (Shell Sort idea):

Logic

1. Initial gap = $(m+n)/2$
2. Compare and swap elements across both arrays
3. Reduce gap each iteration

Tips

- Mention: “This avoids using $O(n)$ additional space.”
 - Most candidates incorrectly use extra array — avoid that
-

8 Question: Find Kth Largest Element (Heap)

What they asked me

“How do you find the Kth largest element efficiently?”

What I said

I used a **min-heap** of size **k**:

Code

```
import heapq

def kthLargest(nums, k):
    heap = nums[:k]
    heapq.heapify(heap)

    for num in nums[k:]:
```

```
        if num > heap[0]:
            heapq.heappreplace(heap, num)

    return heap[0]
```

Time Complexity

$O(n \log k)$

Tips

- They prefer heap solution
 - Avoid sorting full array ($O(n \log n)$)
-

9 Question: Sliding Window Maximum

What they asked me

“Given an array, find max in each window of size k.”

What I said

I used a **deque** to store useful indices.

Code

```
from collections import deque

def maxSlidingWindow(nums, k):
    dq = deque()
    result = []

    for i, num in enumerate(nums):
        if dq and dq[0] == i-k:
            dq.popleft()

        while dq and nums[dq[-1]] < num:
            dq.pop()

        dq.append(i)

        if i >= k-1:
            result.append(nums[dq[0]])

    return result
```

Tips

- $O(n)$ solution using deque → interview favorite
 - Don't use nested loops or sorting → too slow
-

10 Question: Binary Search on Rotated Sorted Array

What they asked me

“Search a number in a rotated array.”

What I said

Use **modified binary search**.

Logic

- One half is always sorted
- Decide where the target lies
- Move left or right accordingly

Code

```
def search(nums, target):
    l, r = 0, len(nums)-1
    while l <= r:
        mid = (l+r)//2
        if nums[mid] == target:
            return mid

        if nums[l] <= nums[mid]:
            if nums[l] <= target < nums[mid]:
                r = mid-1
            else:
                l = mid+1
        else:
            if nums[mid] < target <= nums[r]:
                l = mid+1
            else:
                r = mid-1
    return -1
```

Tips

- Mention: “This maintains $O(\log n)$ complexity.”
- Don't try to linear scan — not acceptable for 2–4 YOE